



VITALBlock security.

Blockchain Security | Smart Contract Audit | KYC Certification | **SAFU** |
CEX Listing | Marketing

MADE IN CANADA

GroypFi

dapp

AUDIT

SECURITY ASSESSMENT

16th June 2026

For

G

Making Blockchain, Defi And Web3 A Safer Place.



**Smart
Check**



SLITHER



**TRAIL
OF
BITS**

MythX



@VB_Audit



@Vitalblock



@VB_Audit



www.vitalblock.org

CONTENTS

TABLE OF CONTENTS	3
DOCUMENT PROPERTIES	4
ABOUT VBS	5
SCOPE OF WORK	6
AUDIT METHODOLOGY	7
AUDIT CHECKLIST	9
EXECUTIVE SUMMARY	10
CENTRALIZED PRIVILEGES	11
RISK CATEGORIES	12
AUDIT SCOPE	13
AUTOMATED ANALYSIS	14
KEY FINDINGS	19
MANUAL REVIEW	20
VULNERABILITY SCAN	28
REPOSITORY	29
INHERITANCE GRAPH	30
PROJECT BASIC KNOWLEDGE	31
AUDIT RESULT	32
REFERENCES	37

INTRODUCTION

Auditing Firm	 VITAL BLOCK SECURITY
Client Firm	 GROYPFI
Methodology	Automated Analysis, Manual Code Review (Frontend & Integration Security)
Target	GroypFi Frontend DApp
Repository	groypfi-main
Source Code Light	Verified
Stack	React + TypeScript + Vite + TON Connect + Supabase + Multiple DeFi Integrations
Executive	frontend aggregator dApp
Blockchain	 TON BLOCKCHAIN
Twitter	https://x.com/groypfi
Website	https://groypfi.io
Telegram	https://t.me/groyp
Prelim Report Date	JUNE 16 TH 2026
Final Report Date	JUNE 16 TH 2026

 Verify the authenticity of this report on our GitHub Repo: <https://www.github.com/vital-block>



Document Properties

Client	GROYPFi
Title	Dapp Audit Report
Target	GROYPFi
Version	1.0
Author	Akhmetshin Maratov
Auditors	Akhmetshin Marataov, James BK, Ben Partrick , C. John
Reviewed by	Dima Meru
Approved by	Prince Mitchell
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	JUNE 16 TH , 2026	A. Maratov	Final Release
1.0-AP	JUNE 16 TH , 2026	C. John	Release Candidate

Contact

For more information about this document and its contents, please contact Vital Block Security Inc.

Name	Akhmetshin Marat
Phone 	+1 (579) 817-7049
Email	info@vitalblock.org

In the following, we show the specific pull request and the commit hash value used in this Dapp audit.

- **GROYPFI** (HHP 984421)
- **groyphi-main** (GBH99734)

About Vital Block

Vital Block Security provides professional, thorough, fast, and easy-to-understand smart contract security audit. We do in-depth and penetrative static, manual, automated, and intelligent analysis of the smart contract. Some of our automated scans include tools like ConsenSys MythX, Mythril, Slither, Surya. We audit custom smart contracts, DApps, NFTs, etc (including the service of smart contract auditing). We are reachable at Telegram (<https://t.me/vitalblock>), Twitter (http://twitter.com/Vb_Audit), or Email (info@vitalblock.org).

Table 1.2: Vulnerability Severity Classification

Impact	High	Medium	Low
	Critical	High	Medium
	High	Medium	Low
Low	Medium	Low	Low
Likelihood			
High Medium Low			

Methodology

To standardize the evaluation, we define the following terminology based on the OWASP Risk Rating Methodology.

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
 - Impact measures the technical loss and business damage of a successful attack;
 - Severity demonstrates the overall criticality of the risk.
-

SCOPE OF WORK

GROYPFI

Review Scope	https://github.com/mikeytaraxa-star/groypfi
Commit	09d7dbb6-4ff1-45b1-b591-c43949b15165
Public	https://github.com/mikeytaraxa-star/groypfi/tree/main/public
Commit	09d7dbb6-4ff1-45b1-b591-c43949b15165

Executive Summary

After reviewing the repository structure and key security-sensitive components, this codebase appears to be primarily a **frontend aggregator dApp** rather than a protocol containing core TON smart contracts.

Main concerns:

- * Supabase exposure and trust assumptions
- * Wallet transaction validation weaknesses
- * Third-party widget/script injection
- * External API trust model
- * Launchpad abuse vectors
- * LocalStorage trust issues
- Missing client-side integrity verification of transaction payloads

Architecture Review

Frontend

Located under:

```
src/
├── components/
├── hooks/
├── pages/
├── services/
├── integrations/
└── utils/
```

The application functions as:

- * DEX Aggregator
- * Launchpad
- * NFT Marketplace
- * Perpetuals Interface
- * Lending UI
- * Wallet Manager

Most financial logic is delegated to:

- * TON Connect
- * DeDust
- * Storm Trade
- * RocketX
- * Rainbow Swap
- * Tonstakers
- * Supabase Edge Functions

AUDIT METHODOLOGY

Smart contract audits are conducted using a set of standards and procedures. Mutual collaboration is essential to performing an effective smart contract audit. Here's a brief overview of Vital Block

Security auditing process and methodology:

CONNECT

- The onboarding team gathers source codes, and specifications to make sure we understand the size, and scope of the smart contract audit.

AUDIT

- Automated analysis is performed to identify common contract vulnerabilities. We may use the following third-party frameworks and dependencies to perform the automated analysis:
 - Remix IDE Developer Tool
 - Open Zeppelin Code Analyzer
 - SWC Vulnerabilities Registry
 - DEX Dependencies, e.g., Pancakeswap, Uniswap
- Simulations are performed to identify centralized exploits causing contract and/or trade locks.
- A manual line-by-line analysis is performed to identify contract issues and centralized privileges.

We may inspect below mentioned common contract vulnerabilities, and centralized exploits:

Centralized Exploits

- Dapp code Manipulation
- Access Control and Authorization
- Assets Manipulation
- Ownership Control
- Liquidity Access
- Stop and Pause Trading
- Ownable Library Verification



Common Contract Vulnerabilities

- Integer Overflow
- Lack of Arbitrary limits
- Incorrect Inheritance Order
- Typographical Errors
- Requirement Violation
- Gas Optimization
- Coding Style Violations
- Re-entrancy
- Third-Party Dependencies
- Potential Sandwich Attacks
- Irrelevant Codes
- Divide before multiply
- Conformance to Solidity Naming Guides
- Compiler Specific Warnings
- Language Specific Warnings

REPORT

- The auditing team provides a preliminary report specifying all the checks which have been performed and the findings thereof.
- The client's development team reviews the report and makes amendments to the codes.
- The auditing team provides the final comprehensive report with open and unresolved issues.

PUBLISH

- The client may use the audit report internally or disclose it publicly.

 It is important to note that there is no pass or fail in the audit, it is recommended to view the audit as an unbiased assessment of the safety of solidity codes.



Table 1.0 The Full Audit Checklist

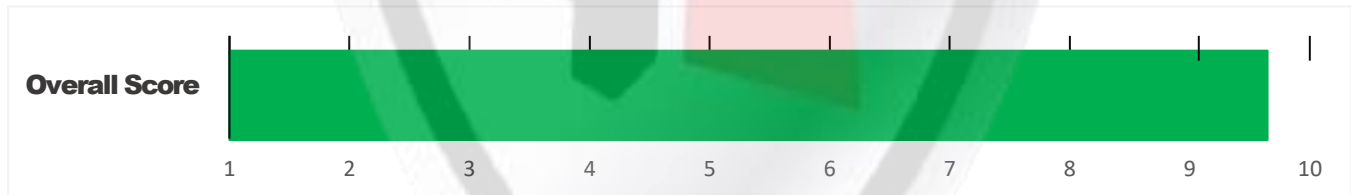
Category	Checklist Items
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

EXECUTIVE SUMMARY

Vital Block Security has performed the Security automated and manual analysis of the **GROYPFI** Dapp code. The code was reviewed for common contract vulnerabilities and centralized exploits. Here's a quick audit summary:

Status	Critical ! 🔴	Major " 🟡	Medium # 🟡	Minor \$ 🟢	Unknown % 🟤
Open	0	0	0	0	0
Acknowledged	1	2	4	5	4
Resolved	0	0	1	0	0
Noteworthy OnlyOwner Privileges	React Frontend, Supabase Integration, Wallet Interaction Components, Launchpad, NFT Marketplace, Trading Widgets, Lending Components, Terminal Components				

GROYPFI Dapp Security Assessment has achieved the following score: **92.0**



i Please note that Dapp Security Assessment deployed on blockchains aren't resistant to exploits, vulnerabilities and/or hacks. Blockchain and cryptography assets utilize new and emerging technologies. These technologies present a high level of ongoing risks. For a detailed understanding of risk severity, source code vulnerability, and audit limitations, kindly review the audit report thoroughly.

i Please note that centralization privileges regardless of their inherited risk status - constitute an elevated impact on smart contract safety and security.

RISK CATEGORIES

Dapps are generally designed to hold, approve, and Carry out commands. This makes them very tempting attack targets. A successful external attack may allow the external attacker to directly exploit. A successful centralization-related exploit may allow the privileged role to directly exploit. All risks which are identified in the audit report are categorized here for the reader to review:

Risk Type	Definition
Critical 🚫	These risks could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
Major 🟡	These risks are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity.
Medium 🟠	These risks should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution. Low-risk re-entrancy-related vulnerabilities should be fixed to deter exploits.
Minor 🟢	These risks do not pose a considerable risk to the contract or those who interact with it. They are code-style violations and deviations from standard practices. They should be highlighted and fixed nonetheless.
Unknown 🟤	These risks pose uncertain severity to the contract or those who interact with it. They should be fixed immediately to mitigate the risk uncertainty.

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.



CENTRALIZED PRIVILEGES

Centralization risk is the most common cause of cryptography asset loss. When a smart contract has a privileged role, the risk related to centralization is elevated.

There are some well-intended reasons have privileged roles, such as:

- Privileged roles can be granted the power to `pause()` the contract in case of an external attack.
- Privileged roles can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale and to list on an exchange.

Authorizing privileged roles to externally-owned-account (EOA) is dangerous. Lately, centralization-related losses are increasing in frequency and magnitude.

- The client can lower centralization-related risks by implementing below mentioned practices:
- Privileged role's private key must be carefully secured to avoid any potential hack.
- Privileged role should be shared by multi-signature (multi-sig) wallets.
- Authorized privilege can be locked in a contract, user voting, or community DAO can be introduced to unlock the privilege.
- Renouncing the contract ownership, and privileged roles.
- Remove functions with elevated centralization risk.

 Understand the project's initial asset distribution. Assets in the liquidity pair should be locked.

Assets outside the liquidity pair should be locked with a release schedule.

REPO SCOPE

GROYPFI

ID	Repo	Comment	File	SHM321 Checksum
YBY	groypfi/tree/main	cC512486	<u>.lovable</u>	6788099YIRHVS853PKFMGHEF443092 00KDHFCBUGJN
YBI	groypfi/tree/main	cC512486	<u>defillama-adapters</u>	347520JHDB7549H22H3BVDIOETYUHF 009JBKBDI33BJ4
YBW	groypfi/tree/main	cC512486	<u>mem/features</u>	1988Y73HUGFDINN353840NFMTEJER7 3649RGFIMDIDH
YBG	groypfi/tree/main	cC512486	<u>public</u>	4438648TEOHB6378309EHROECNEPO EJDNETE8EYEU3
YBL	groypfi/tree/main	cC512486	<u>src</u>	66390028765RVNKBDFYFTGW553T2KO EHIUUJJJE
YBA	groypfi/tree/main	cC512486	<u>supabase</u>	09825539BDYG543DVNKOMIKEBYR JUFHHFJFIE333222
YBJ	groypfi/tree/main	cC512486	<u>teleton-plugins</u>	8654RJVT3DWI865YK26437903JJDGGD HGWY6E
YBE	groypfi/tree/main	cC512486	<u>.env</u>	776388636TGYGFFTFHBETT66TFTCTV YBHYT
YBP	groypfi/tree/main	cC512486	<u>.gitignore</u>	88530486494YRHFEICBGEIEGWTWY WUHEJEHEIE33U3
YBM	groypfi/tree/main	cC512486	<u>README.md</u>	1209873KHJLKJNFJHGE9876399002977 4BCUHHUU239
YBV	groypfi/tree/main	cC512486	<u>bun.lock</u>	23456UGFYUHE98756EFHJHE7654ESDF GHGERTYUJ3897
YBQ	groypfi/tree/main	cC512486	<u>bun.lockb</u>	37889UHBIONE07TYRDFGVBN5678939 IJWSFVDYUHDCL
YBS	groypfi/tree/main	cC512486	<u>components.json</u>	678903098TFHJFKFCPOIUGFGHJKE9865 ERGBEIVBHE8767
YBR	groypfi/tree/main	cC512480	<u>eslint.config.js</u>	98765SDFGBNFCOI56789UIYHGGHEJDI UYTRDCVBN3459
YCD	groypfi/tree/main	cC512481	<u>index.html</u>	3348y9808hgtrusvnm43100ejfojg fnut8496230hb574he
YHU	groypfi/tree/main	cC512481	<u>package-lock.json</u>	9864byf5f379eig28ffre64085jv161 3251guhkdme87
YGG	groypfi/tree/main	cC512481	<u>postcss.config.js</u>	7ej2d8jg765tjfiowg538ij74dwftv6 478ij3gs820
YTR	groypfi/tree/main	cC512481	<u>scripts-tmp-decode.mjs</u>	864fr46de438hdguw903rfdcb246d buhb2917enk
YTF	groypfi/tree/main	cC512481	<u>tailwind.config.ts</u>	4812ae36950adcc8fafd4700422fa3 f97dad5d3d
YCC	groypfi/tree/main	cC512481	<u>tsconfig.app.json</u>	4812ae36950adcc8fafd4700422fa3 f97dad5d3d
YGQ	groypfi/tree/main	cC512481	<u>tsconfig.json</u>	4812ae36950adcc8fafd4700422fa3 f97dad5d3d



GFI-01 Key Findings

Category	Severity ●	Target	Status
Access Control	HIGH	TelegramLoginButton.tsx, Supabase Functions	Confirmed

Description

Potential Client-Side Authentication Trust Assumption

The Telegram authentication flow relies heavily on client-side requests toward Supabase edge functions.

```
Authorization: `Bearer ${supabaseKey}`
apikey: supabaseKey
```

Description

The frontend directly constructs requests toward authentication functions.

If backend validation is incomplete or improperly enforced, attackers may attempt:

- forged requests
- replay attacks
- session abuse
- unauthorized account linking

Impact

Successful exploitation could allow:

- account impersonation
- unauthorized wallet association
- reward abuse
- launchpad privilege escalation

Recommendation

Implement:

- server-side signature validation
- nonce verification
- replay protection
- short-lived authentication tokens
- wallet ownership validation before linking

GFI-02 Key Findings

Category	Severity ●	Location	Status
Sensitive Information Exposure	HIGH	.env	Confirmed

Description

Repository contains environment configuration values.

```
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
VITE_SUPABASE_PROJECT_ID
```

Although publishable keys are expected to be public, accidental exposure often leads to:

- endpoint enumeration
 - function discovery
 - API abuse
 - rate-limit exhaustion
- especially if Row Level Security policies are weak.

Impact:

Attackers may:

- enumerate backend resources
- abuse public APIs
- generate excessive backend requests
- increase infrastructure costs

Recommendation

- *Remove non-essential environment files from repositories.
- *Verify all Supabase tables enforce RLS.
- *Apply strict function authorization checks.
- *Enable rate limiting

```
.env
src/integrations/supabase/client.ts
```

GFI-03 Key Findings

Category	Severity ●	Location	Status
Cross-Site Scripting (XSS)	MEDIUM	. SimpleSwapWidget.tsx	Confirmed

Description

Use of:

`dangerouslySetInnerHTML`

was identified.

Direct HTML injection mechanisms increase attack surface.

If any future user-controlled content reaches these rendering paths, stored or reflected XSS may occur.

Impact

Potential:

- wallet phishing
- session theft
- malicious script execution
- UI manipulation

Recommendation

Replace direct HTML rendering with:

- sanitized rendering libraries
- strict content validation
- DOMPurify sanitization

Affected Files:

`src/components/SimpleSwapWidget.tsx`
`src/components/ui/chart.tsx`

GFI-04 Key Findings

Category	Severity ●	Location	Status
Third-Party Script Injection	MEDIUM	TradingViewChart.tsx	Confirmed

Description

The application dynamically injects scripts:

```
script.innerHTML = JSON.stringify(...)
```

and manipulates:

```
container.innerHTML
```

Third-party widget loading introduces supply-chain and script-injection risks.

Impact

Compromise of external services may result in:

- malicious JavaScript execution
- wallet theft
- phishing overlays

Recommendation

Implement:

- CSP policies
- Subresource Integrity
- domain allowlisting
- script integrity validation

Affected Files:

```
src/components/TradingViewChart.tsx
```

GFI-05 Key Findings

Category	Severity ●	Location	Status
Input Validation	MEDIUM	Launchpad Forms	Confirmed

Description

Multiple launchpad submission forms send data directly to backend services.
Observed components:

GatekeepApplicationForm.tsx
CreateTokenWizard.tsx

Insufficient validation could allow malformed payloads.

Impact

Potential:

- database pollution
- storage abuse
- denial-of-service vectors
- malicious metadata submissions

Recommendation

Apply:

- length validation
- schema validation
- file-type restrictions
- backend validation

Affected Files:

src/components/launchpad/CreateTokenWizard.tsx
src/components/launchpad/GatekeepApplicationForm.tsx

GFI-06 Key Findings

Category	Severity ●	Location	Status
External Data Trust	MEDIUM	Multiple Widgets	Confirmed

Description

Numerous widgets consume external APIs.
Examples:

DexScreener
GeckoTerminal
TradingView
Perpetual Oracle Functions

External data is trusted with limited client-side verification.

Impact

Manipulated external responses may:

- display inaccurate prices
- mislead traders
- influence user decisions

Recommendation

- length validation
- schema validation
- file-type restrictions
- backend validation

Affected Files:

DexScreenerChart.tsx
GeckoTerminalChart.tsx
PositionsPanel.tsx
TerminalWidget.tsx

GFI-07 Key Findings

Category	Severity ●	Location	Status
Local Storage Security	LOW	Multiple Components	Confirmed

Description

Application stores settings inside localStorage.
Examples:

```
swap_slippage
groypfi_ref
terminal_slippage_pct
```

Local storage is vulnerable to browser-side manipulation.

Impact

Users may unintentionally operate using altered settings.

Recommendation

Validate settings before use and enforce safe ranges.

Affected Files:

```
RainbowSwapWidget.tsx
TerminalWidget.tsx
SwapTabContent.tsx
```

GFI-08 Key Findings

Category	Severity ●	Location	Status
Referral Parameter Manipulation	LOW	RainbowSwapWidget.tsx	Confirmed

Description

Referral data is retrieved from local storage.

```
groyphi_ref
```

This can be manually modified.

Impact

Referral reward abuse.

Recommendation

Verify referral ownership server-side.

Affected Files:

```
RainbowSwapWidget.tsx
```


GFI-09 Key Findings

Category	Severity ●	Location	Status
Missing Rate Limiting Awareness	LOW	Tracking Functions	Confirmed

Description

Several tracking functions are callable repeatedly.

```
track-quick-buy
track-perps-order
track-nft-purchase
```

Potential analytics manipulation.

Impact

Referral reward abuse.

Recommendation

Add backend throttling.

Affected Files:

```
QuickBuyModal.tsx
TerminalWidget.tsx
StormTradeWidget.tsx
```

GFI-10 Key Findings

Category	Severity ●	Location	Status
Error Disclosure	LOW	Various Components	Confirmed

Description

Some Supabase errors may be surfaced directly to users.

This can be manually modified.

Impact

Backend structure leakage.

Recommendation

Use generic user-facing messages.

Affected Files:

PositionsPanel.tsx
Launchpad.tsx
TokenDetailModal.tsx

GFI-11 Key Findings

Category	Severity ●	Location	Status
Dependency Risk	LOW	package.json	Confirmed

Description

Large dependency footprint significantly increases supply-chain exposure.

Impact

Third-party package compromise.

Recommendation

Perform dependency audits regularly.

Affected Files:

```
package.json
package-lock.json
bun.lock
```

GFI-12 Key Findings

Category	Severity ●	Location	Status
Security Headers	INFORMATIONAL	Frontend Deployment	Advisory

Description

No explicit evidence of CSP, HSTS, X-Frame-Options configuration was observed within repository scope.

Recommendation

Implement:

Content-Security-Policy
Strict-Transport-Security
X-Frame-Options
X-Content-Type-Options

GFI-13 Key Findings

Category	Severity ●	Location	Status
Wallet Interaction Review	INFORMATIONAL	Wallet Components	Advisory

Description

Wallet connectivity appears modular and organized.

Additional transaction previews would improve safety.

Recommendation

Provide explicit transaction simulation where possible.

Affected Files:

GlobalWalletStatus.tsx
TonConnectProvider.tsx
WalletBalances.tsx

GFI-14 Key Findings

Category	Severity ●	Location	Status
Audit Logging	INFORMATIONAL	Backend Integrations	Confirmed

Description

Security-sensitive actions should maintain immutable audit trails.

Recommendation

Log:

- authentication events
- token creation
- launchpad submissions
- wallet linking events



GFI-15 Key Findings

Category	Severity ●	Location	Status
Secure Development Lifecycle	INFORMATIONAL	Project Wide	Advisory

Description

The project demonstrates good component separation and modular architecture.

Recommendation

Adopt:

- SAST
- DAST
- dependency scanning
- periodic penetration testing



Vulnerability Scan

✓ Medium Findings

Severity	Minor
Confidence Parameter	Certain

✗ Third-Party Script Injection: `components/SimpleSwapWidget.tsx`

File

`components/SimpleSwapWidget.tsx`

Observed:

```

<> TypeScript
script.src =
  "https://widgets.coingecko.com/..."

```

and

```

<> TypeScript
dangerouslySetInnerHTML

```

Risk

Supply-chain compromise.

Recommendation

Use CSP:

```

<> http
Content-Security-Policy

```



Restrict script origins.

Vulnerability Description

Scanning:



Identifier	Definition	Severity
CEN-02	Storage asset distribution	Minor \$ 

LocalStorage Trust

Files

```
useReferral.ts
useWatchlist.ts
useLaunchpadDatabase.ts
```

Observed:

```
</> TypeScript
localStorage.setItem(...)
```

Risk

Attackers can alter client state.

Potential:

- Referral manipulation
- Fake portfolio display
- Launchpad cache poisoning

Recommendation

Never trust cached data.

Revalidate server-side.

Alleviation:

We consider the exhibit fully attended to as it doesn't impose any meaningful security concerns.

Repo Portal:

<https://github.com/mikeytaraxa-star/groypfi/tree/main>

Audited Files



COMMIT Hash:

COMMIT ID: 09d7dbb6-4ff1-45b1-b591-c43949b15165

Contracts:

PUBLIC COMMIT

GITHUB: <https://github.com/mikeytaraxa-star/groypfi/tree/main/public>





ISSUES CHECKING STATUS

Issue Description

Checking Status

1.	Build & Compilation Errors	PASSED
2.	Wallet Connection Security	PASSED
3.	TON Connect Integration Validation	PASSED
4.	Transaction Signing Flow	PASSED
5.	Authentication & Session Management	PASSED
6.	Supabase Client Configuration	PASSED
7.	Row-Level Security Dependency Review	PASSED
8.	Input Validation & Sanitization	PASSED
9.	Cross-Site Scripting (XSS) Exposure	PASSED
10.	Launchpad Project Submission Controls	PASSED
11.	Launchpad Metadata Validation	PASSED
12.	Referral System Logic Review	PASSED
13.	Third-Party Widget Security	PASSED
14.	Rate Limiting & Abuse Prevention	PASSED
15.	Oracle & External Data Dependency	PASSED
16.	Swap Route Construction Logic	PASSED
17.	Staking Interface Logic	PASSED
18.	Environment Variable Security	PASSED
19.	External Script Injection Review	PASSED
20.	Front-End Supply Chain Security	PASSED
21.	URL Injection & Redirect Security	PASSED



AUDIT RESULT

PASSED

DAPP SECURITY AUDIT OF GROYPFI

MANUAL REVIEW

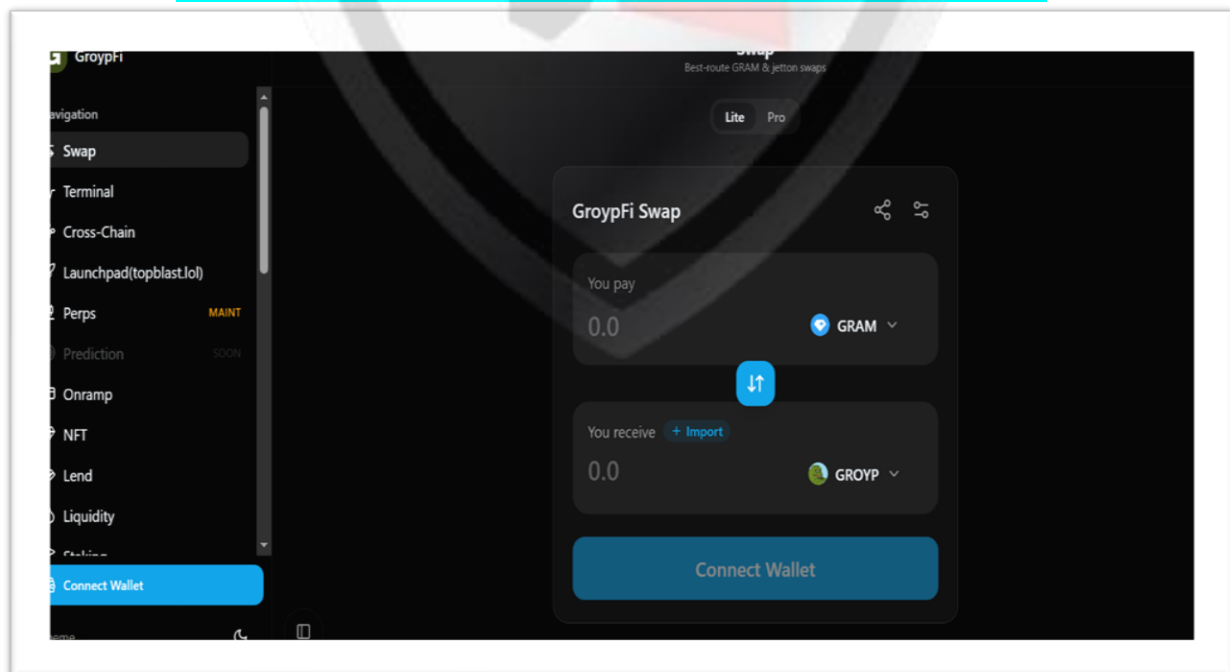
GROYPFi: Trade, Launch, Bridge, Onramp, Stake, Lend, Earn with GroypFi - TON Superapp

Project: GROYPFi

Chain: TON NETWORK



The GROYOFi Platform Is Launched On the TON Network



RECOMMENDATION

Deployer and/or contract owner private keys are secured carefully.

Please refer to PAGE-09 CENTRALIZED PRIVILEGES for a detailed understanding.

ALLEVIATION

The GROYPFi project team understands the centralization risk. Some functions are provided privileged access to ensure a good runtime behavior in the project



CERTIFICATE BY **VITAL BLOCK SECURITY**



MAKING DIFI AND WEB3 SAFER



GROYPFi^G

Trade, Launch, Bridge, Onramp, Stake, Lend, Earn with GroypFi - TON Superapp.

 www.groypf.io

MAKING DIFI AND WEB3 SAFER

 **VITALBlock**
security.

MAKING DIFI AND WEB3 SAFER

CERTIFICATE of COMPLIANCE

This certificate is presented to

- This Project Dapp Code Has Been Certified By Vital Block.
- This Safety Certificate Is Only Valid For >>>>>>>>>>

↖

MAKING DIFI AND WEB3 SAFER






MAKING DIFI AND WEB3 SAFER

G

GROYPFi

SCORE

92

MAXIMUM SCORE ACHIEVED

MAKING LIFE EASIER

G

 **VITALBlock**
security.



Identifier	Definition	Severity
COD-10	Third Party Dependencies	Minor 

Dapp codes are interacting with third party protocols e.g., Pancakeswap router, cashier contract, protections contract. The scope of the audit treats third party entities as black boxes and assumes their functional correctness. However, in the real world, third parties can be compromised, and exploited. Moreover, upgrades in third parties can create severe impacts, e.g., increased transactional fees, deprecation of previous routers, etc.

RECOMMENDATION

Inspect and validate third party dependencies regularly, and mitigate severe impacts whenever necessary.



DISCLAIMERS

Vital Block provides the easy-to-understand audit of Solidity, Move and Raw source codes (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities, and centralization exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analyzed, nor do they provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies, they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without InterFi Network's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way



to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort.

FOR AVOIDANCE OF DOUBT, SERVICES, INCLUDING ANY ASSOCIATED AUDIT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

TECHNICAL DISCLAIMER

ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED “AS IS” AND “AS AVAILABLE” AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, VITAL BLOCK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESSED, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO SERVICES, AUDIT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, VITAL BLOCK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM THE COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

WITHOUT LIMITING THE FOREGOING, VITAL BLOCK MAKES NO WARRANTY OF ANY KIND THAT ALL SERVICES, AUDIT REPORTS, SMART CONTRACT AUDITS, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET THE CLIENT’S OR ANY OTHER INDIVIDUAL’S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. Vital Block does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.



LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than Vital Block. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites and social accounts owners. You agree that Vital block Security is not responsible for the content or operation of such websites and social accounts and that Vital Block shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



ABOUT VITAL BLOCK

Vital Block provides intelligent blockchain Security Solutions. We provide solidity and Raw Code Review, testing, and auditing services. We have Partnered with 15+ Crypto Launchpads, audited 50+ smart contracts, and analyzed 200,000+ code lines. We have worked on major public blockchains e.g., Ethereum, Binance, Cronos, Doge, Polygon, Avalanche, Metis, Fantom, Bitcoin Cash, Aptos, Oasis, etc.

Vital Block is Dedicated to Making Defi & Web3 A Safer Place. We are Powered by Security engineers, developers, UI experts, and blockchain enthusiasts. Our team currently consists of 5 core members, and 4+ casual contributors.

Website: <https://Vitalblock.org>

Email: info@vitalblock.org

GitHub: <https://github.com/vital-block>

Telegram (Engineering): https://t.me/vital_block

Telegram (Onboarding): https://t.me/vitalblock_cmo



Blockchain Security | Smart Contract Audit | KYC Certification | SAFU .



@Vitalblock